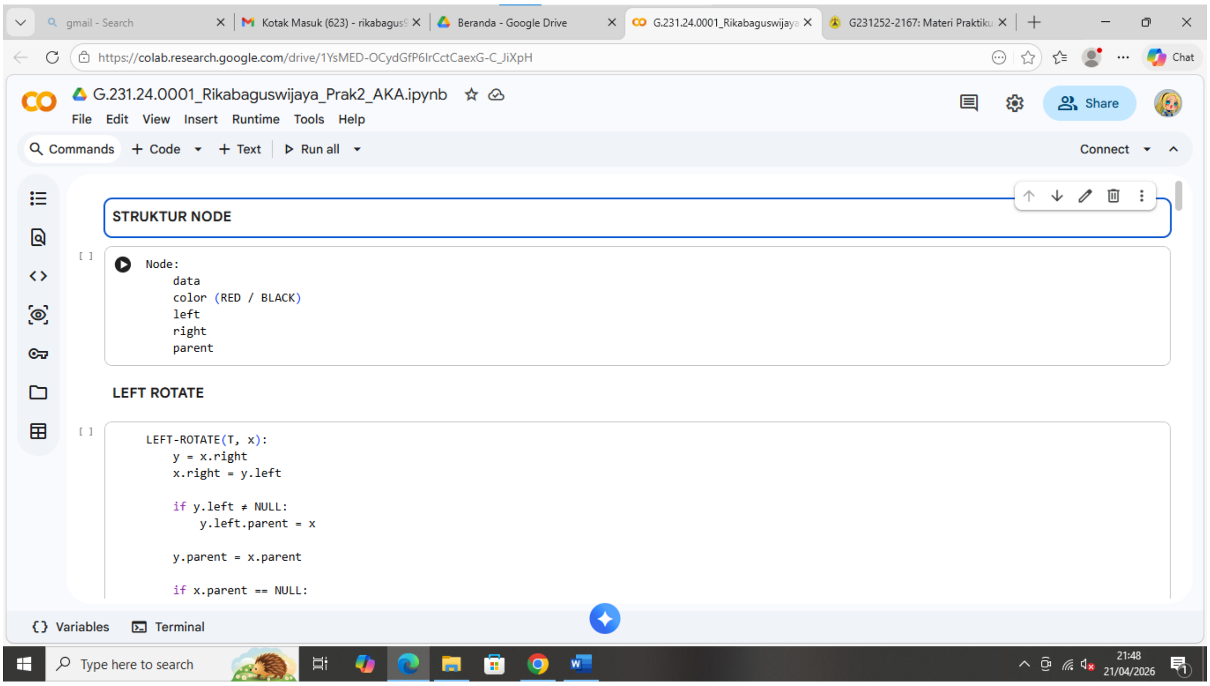
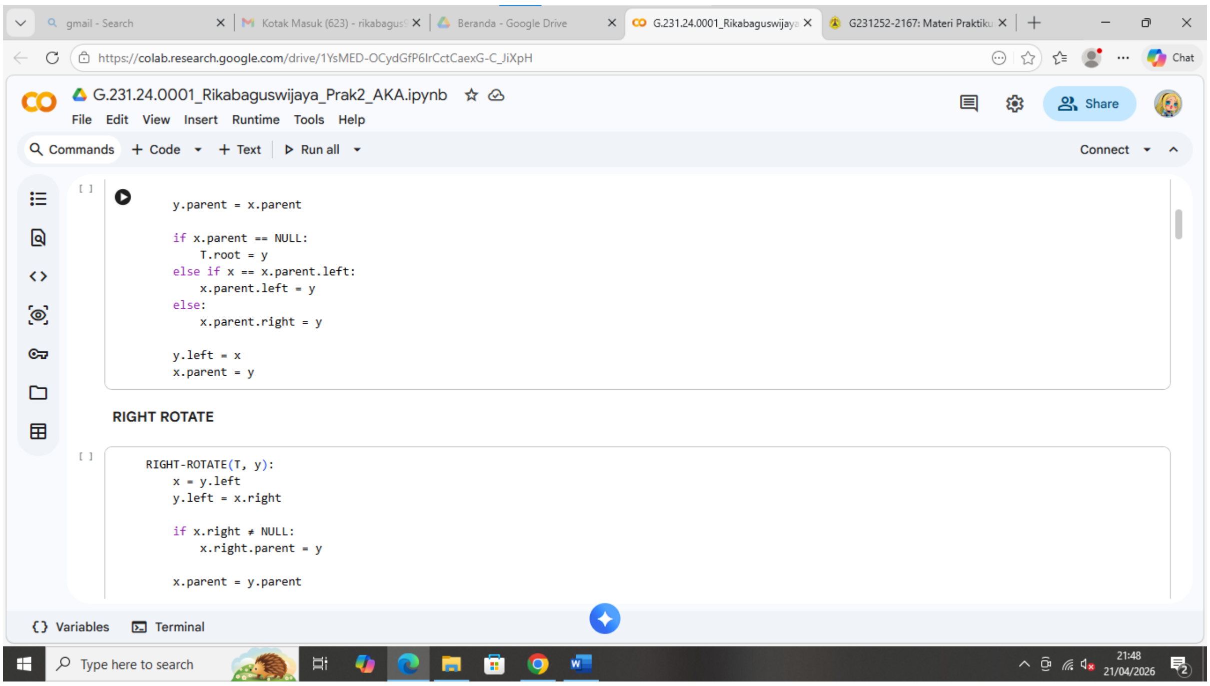
Nama : Rika Bagus Wijaya
Nim : G.231.24.0001
Praktikum 2

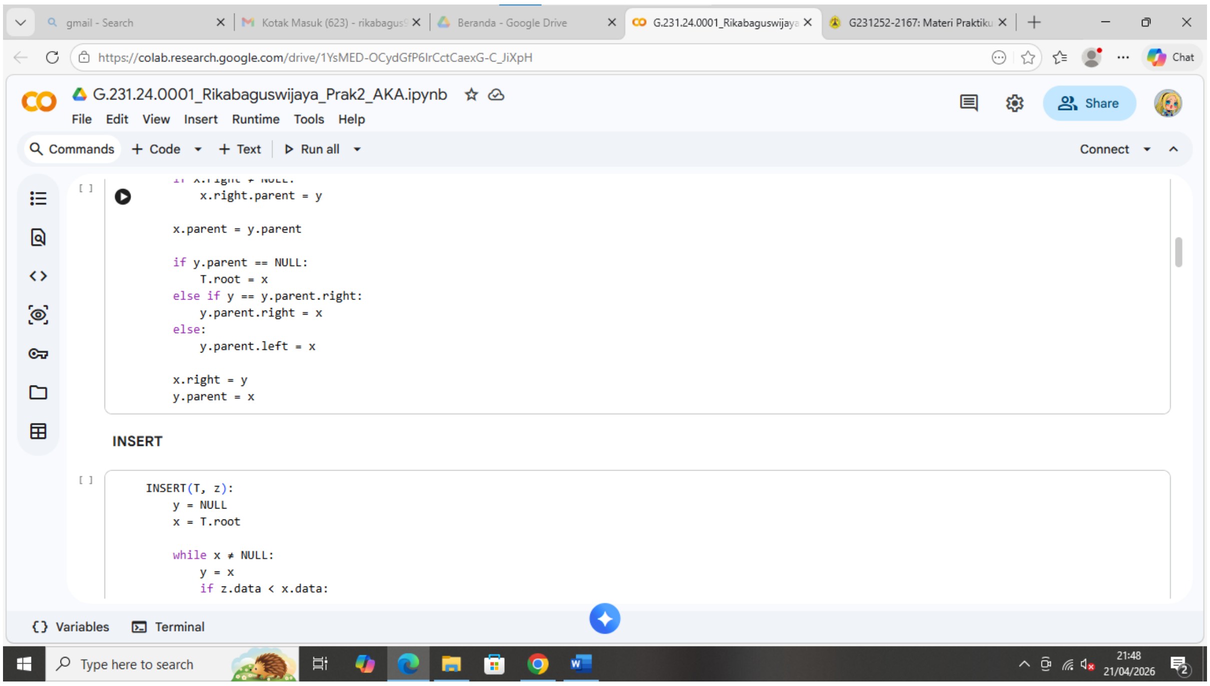
Tugas mencari metode pencarian

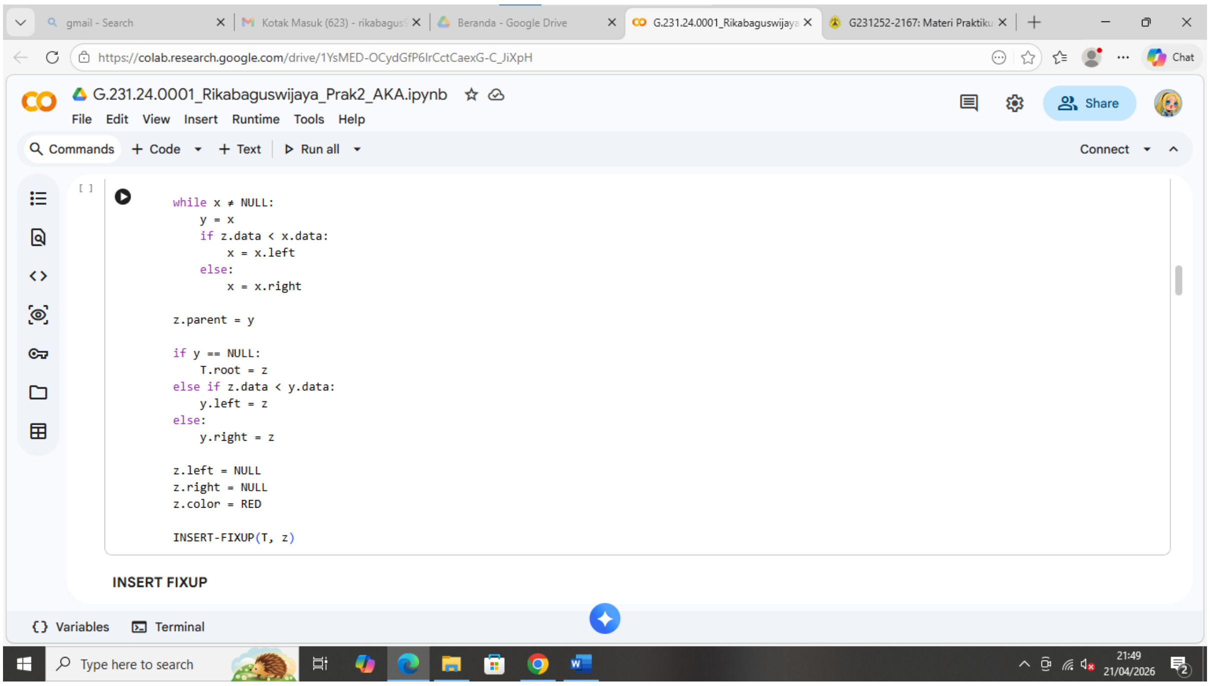
1. buatlah pseudocode dan koding untuk algoritma red-black tree sesuai dengan ketentuan rulenya.
2. analisa worst case, average case

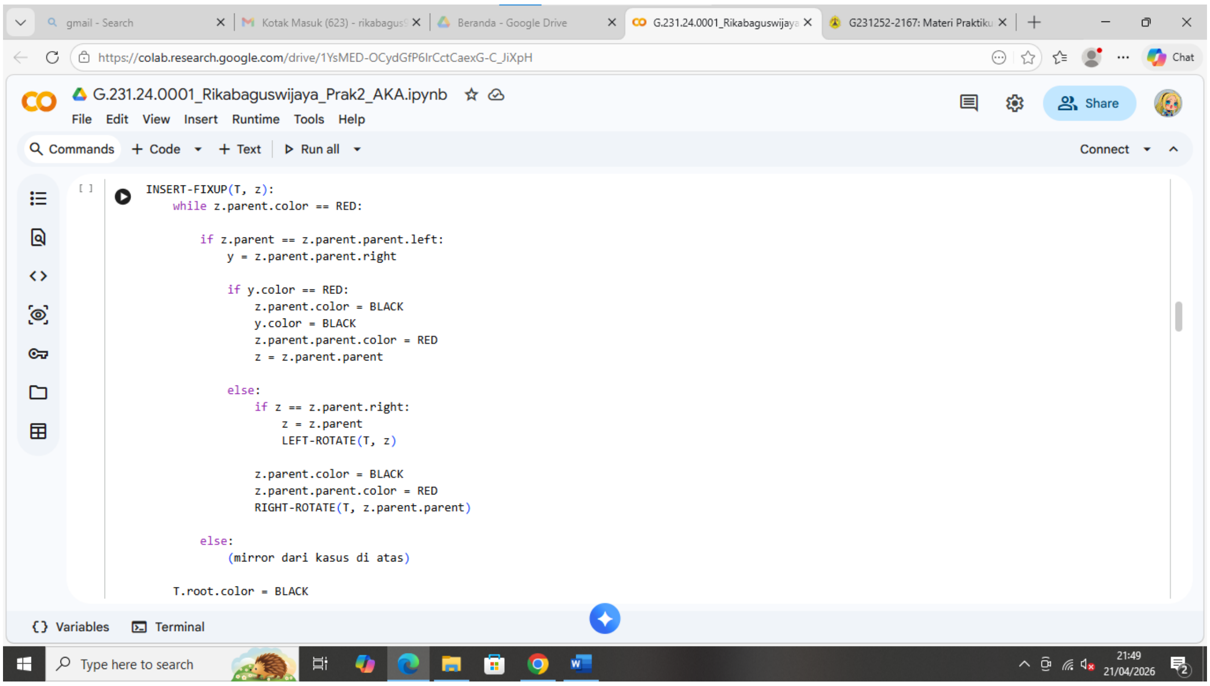
Link kode Pyton saya: https://colab.research.google.com/drive/1YsMED-OCydGfP6lrCctCaexG-C_JiXpH?usp=sharing



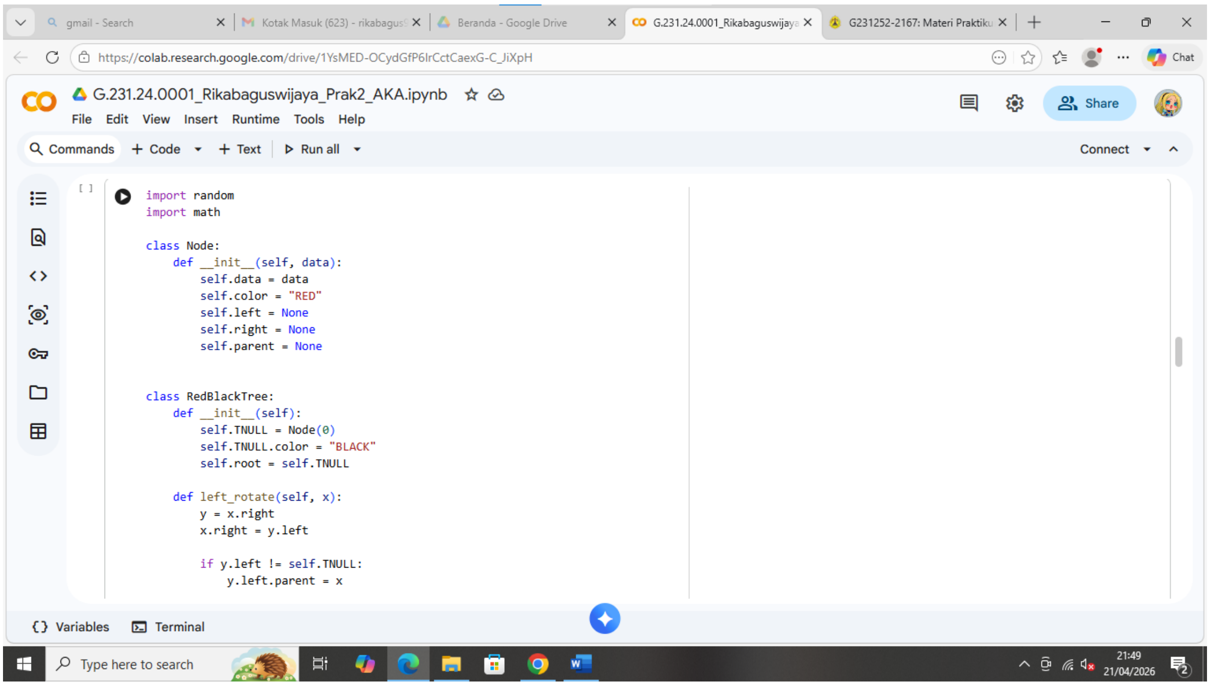



gmail - Search
Kotak Masuk (623) - rikabagus
Beranda - Google Drive
G.231.24.0001_Rikabaguswijaya
G231252-2167: Materi Praktiku
https://colab.research.google.com/drive/1YsMED-OCydGfP6lrCctCaexG-C_JiXpH
Chat
G.231.24.0001_Rikabaguswijaya_Prak2_AKA.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
Connect
x.right.parent = y
x.parent = y.parent
if y.parent == NULL:
T.root = x
else if y == y.parent.right:
y.parent.right = x
else:
y.parent.left = x
x.right = y
y.parent = x
INSERT
INSERT(T, z):
y = NULL
x = T.root
while x ≠ NULL:
y = x
if z.data < x.data:
Variables
Terminal
Type here to search
21:48
21/04/2026

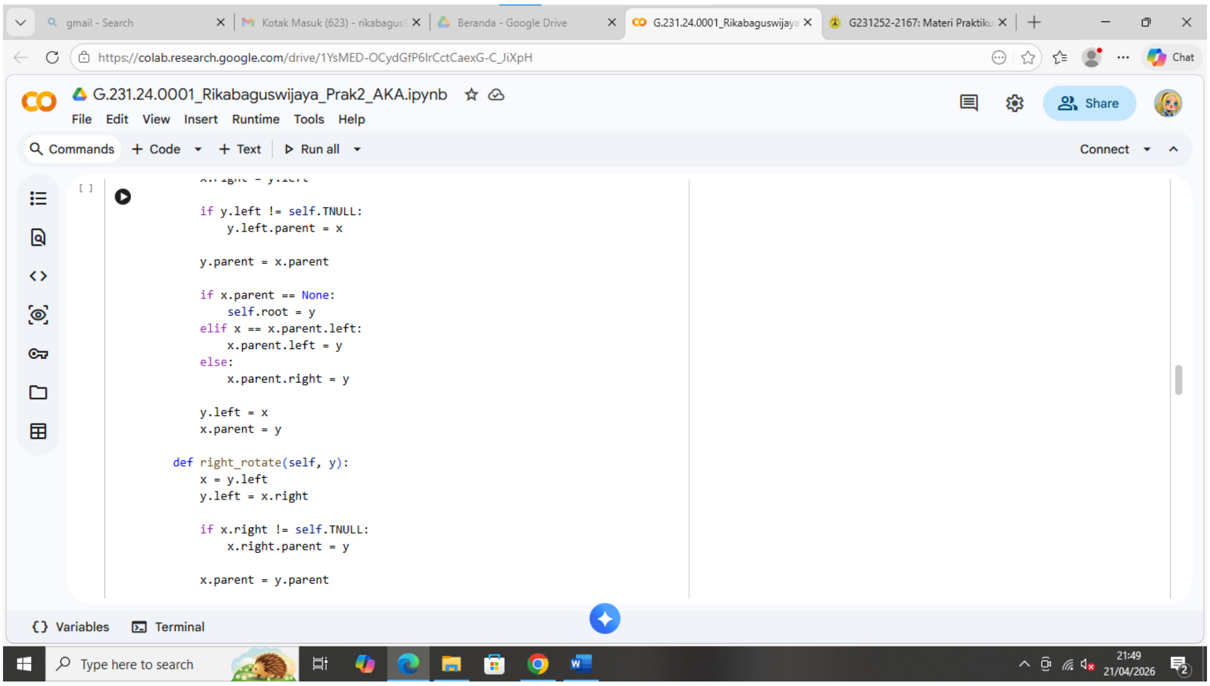

gmail - Search
Kotak Masuk (623) - rikabagus
Beranda - Google Drive
G.231.24.0001_Rikabaguswijaya
G231252-2167: Materi Praktiku
https://colab.research.google.com/drive/1YsMED-OCydGfP6lrCctCaexG-C_JiXpH
Chat
G.231.24.0001_Rikabaguswijaya_Prak2_AKA.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
Connect
while x ≠ NULL:
y = x
if z.data < x.data:
x = x.left
else:
x = x.right
z.parent = y
if y == NULL:
T.root = z
else if z.data < y.data:
y.left = z
else:
y.right = z
z.left = NULL
z.right = NULL
z.color = RED
INSERT-FIXUP(T, z)
INSERT FIXUP
Variables
Terminal
Type here to search
21:49
21/04/2026



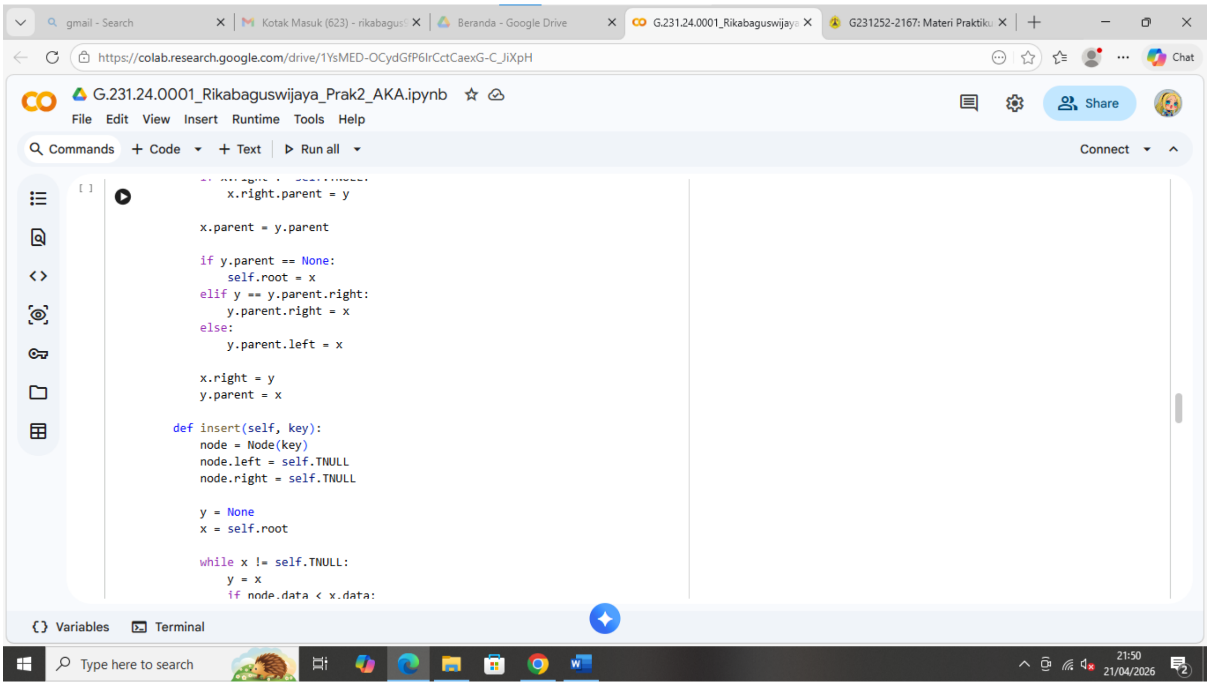

```
INSERT-FIXUP(T, z):
    while z.parent.color == RED:

        if z.parent == z.parent.parent.left:
            y = z.parent.parent.right

            if y.color == RED:
                z.parent.color = BLACK
                y.color = BLACK
                z.parent.parent.color = RED
                z = z.parent.parent

            else:
                if z == z.parent.right:
                    z = z.parent
                    LEFT-ROTATE(T, z)

                z.parent.color = BLACK
                z.parent.parent.color = RED
                RIGHT-ROTATE(T, z.parent.parent)

        else:
            (mirror dari kasus di atas)

    T.root.color = BLACK
```



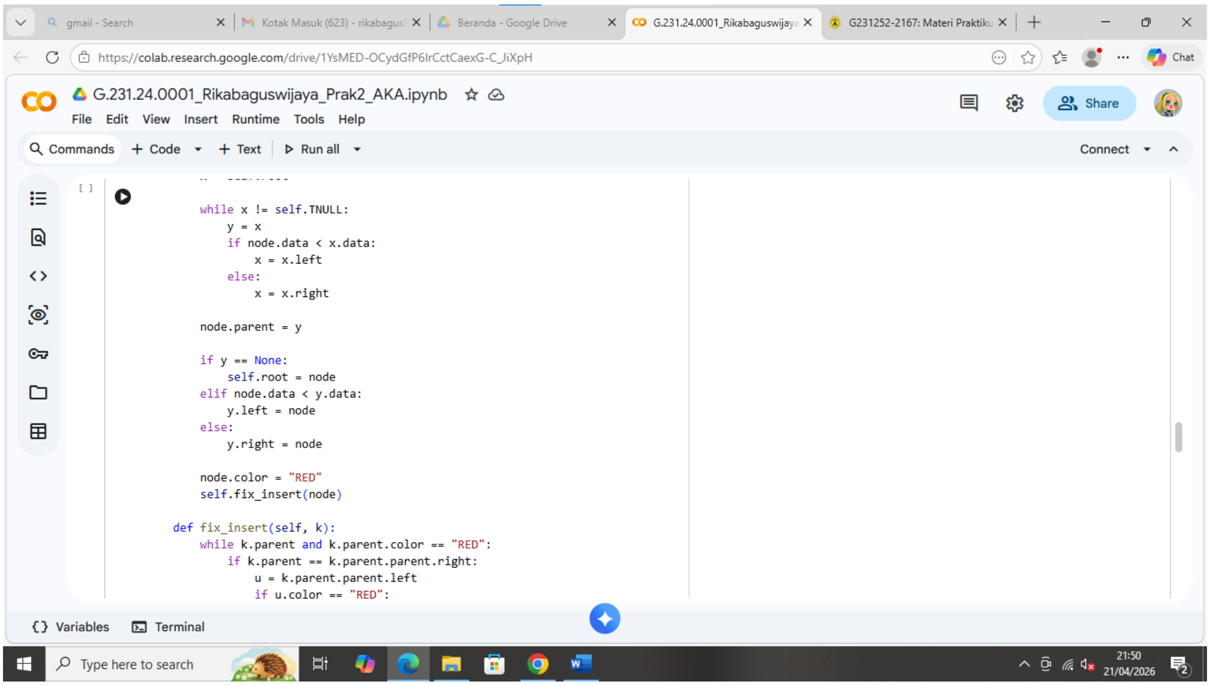

```
import random
import math

class Node:
    def __init__(self, data):
        self.data = data
        self.color = "RED"
        self.left = None
        self.right = None
        self.parent = None


class RedBlackTree:
    def __init__(self):
        self.TNULL = Node(0)
        self.TNULL.color = "BLACK"
        self.root = self.TNULL

    def left_rotate(self, x):
        y = x.right
        x.right = y.left

        if y.left != self.TNULL:
            y.left.parent = x
```



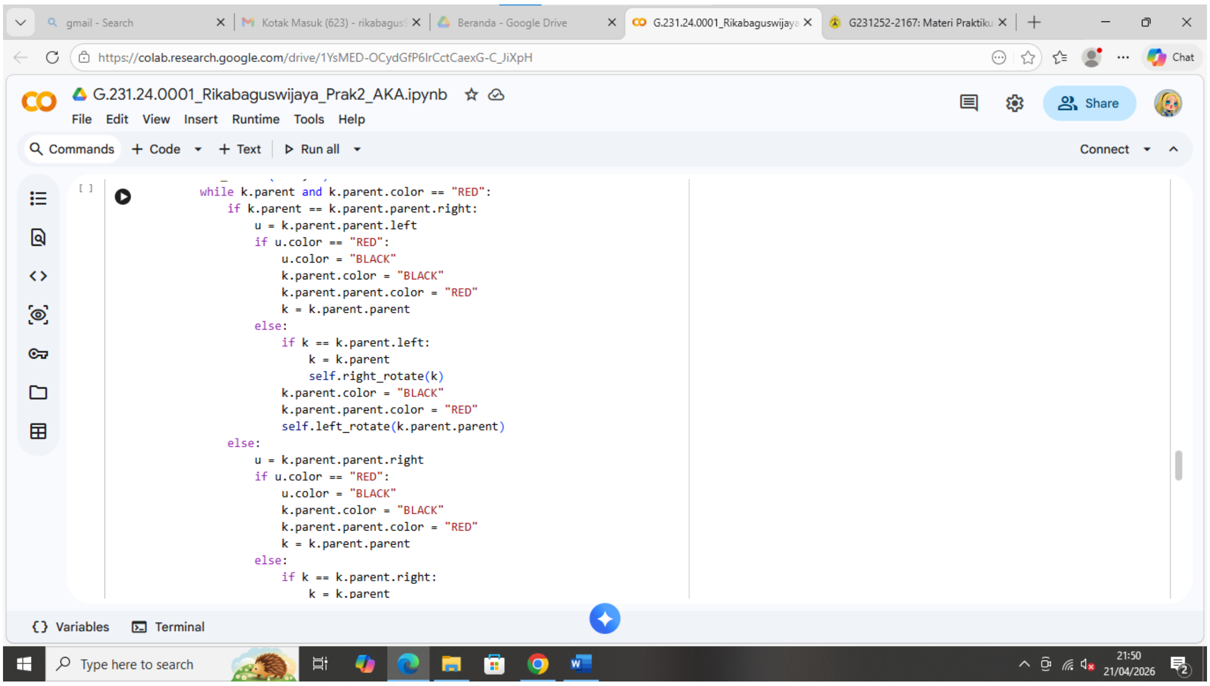

```
        x.right = y.left

        if y.left != self.TNULL:
            y.left.parent = x

        y.parent = x.parent

        if x.parent == None:
            self.root = y
        elif x == x.parent.left:
            x.parent.left = y
        else:
            x.parent.right = y

        y.left = x
        x.parent = y

    def right_rotate(self, y):
        x = y.left
        y.left = x.right

        if x.right != self.TNULL:
            x.right.parent = y

        x.parent = y.parent
```



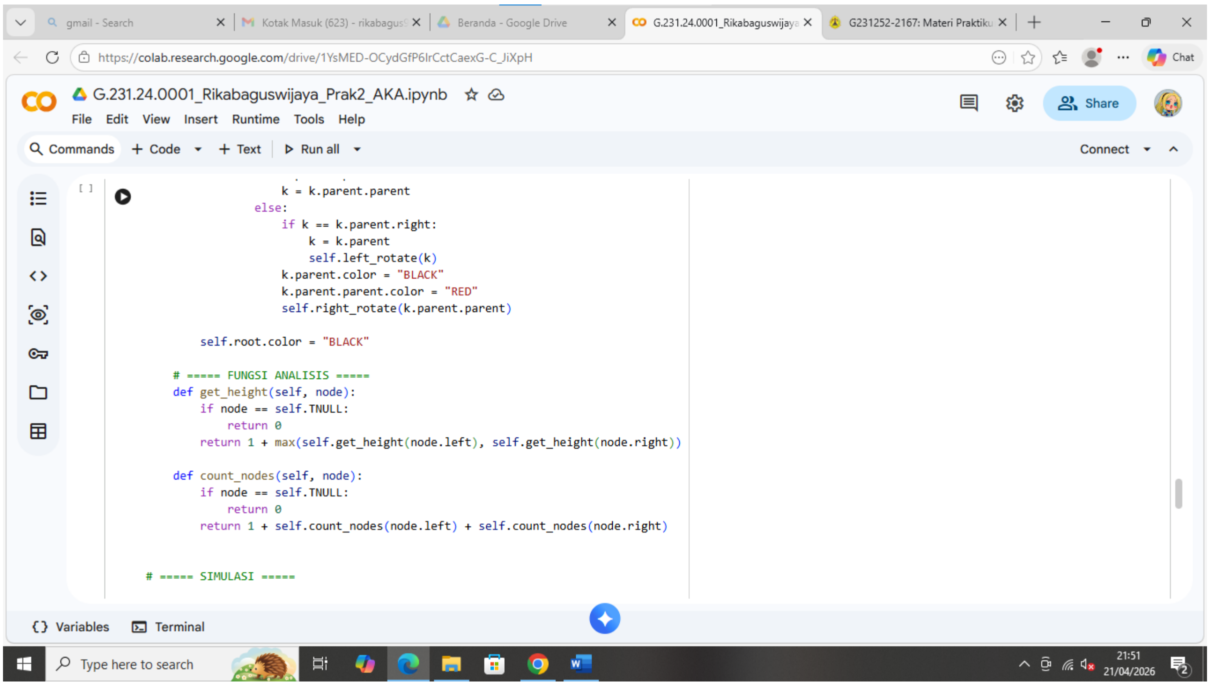

```
        if x.right != self.TNULL:
            x.right.parent = y

        x.parent = y.parent

        if y.parent == None:
            self.root = x
        elif y == y.parent.right:
            y.parent.right = x
        else:
            y.parent.left = x

        x.right = y
        y.parent = x

    def insert(self, key):
        node = Node(key)
        node.left = self.TNULL
        node.right = self.TNULL

        y = None
        x = self.root

        while x != self.TNULL:
            y = x
            if node.data < x.data:
```

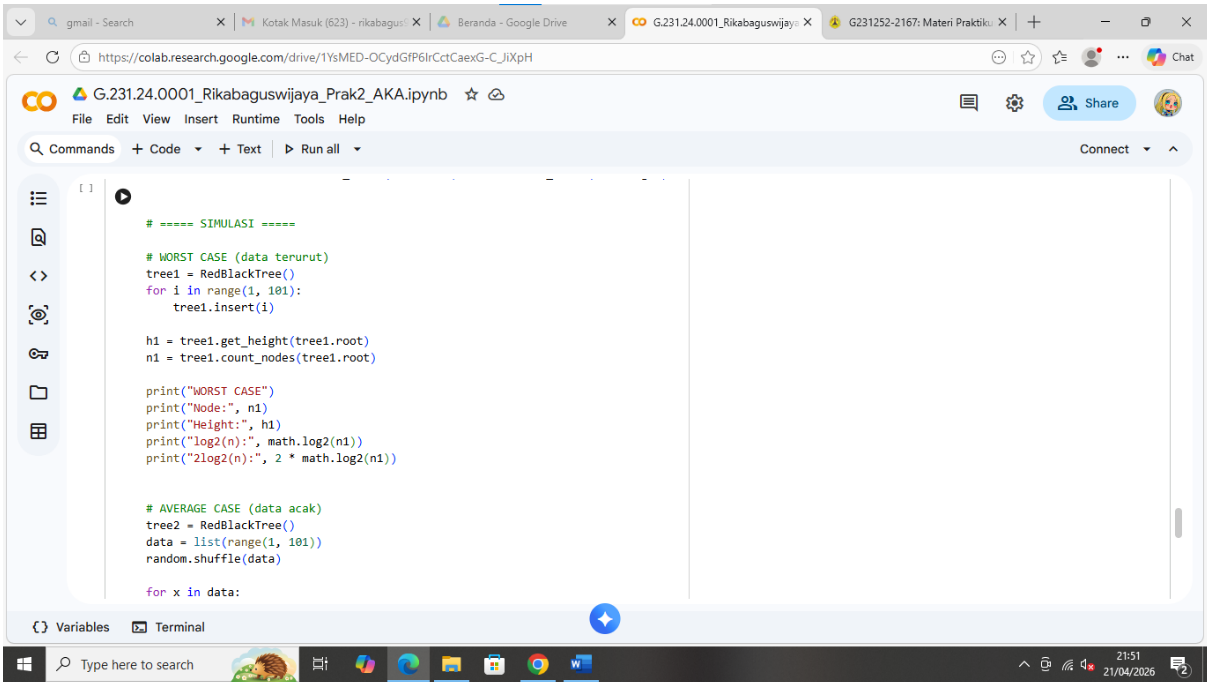


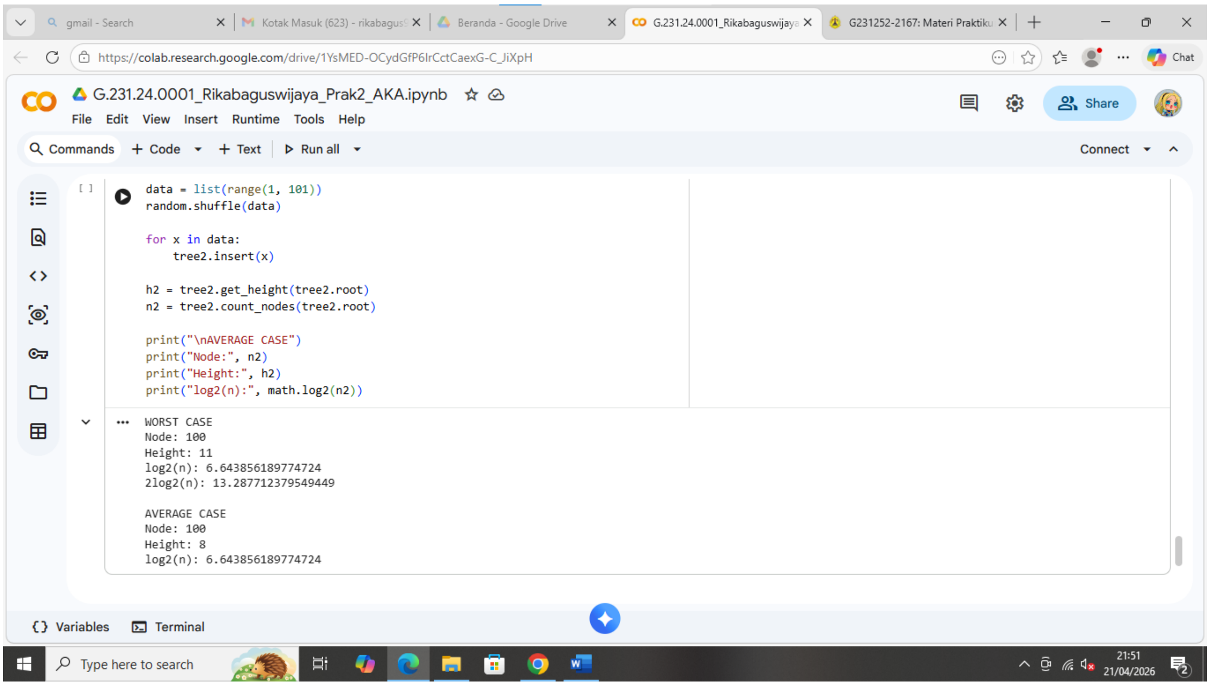
```
        while x != self.TNULL:
            y = x
            if node.data < x.data:
                x = x.left
            else:
                x = x.right

        node.parent = y

        if y == None:
            self.root = node
        elif node.data < y.data:
            y.left = node
        else:
            y.right = node

        node.color = "RED"
        self.fix_insert(node)

    def fix_insert(self, k):
        while k.parent and k.parent.color == "RED":
            if k.parent == k.parent.parent.right:
                u = k.parent.parent.left
                if u.color == "RED":
```



```
        while k.parent and k.parent.color == "RED":
            if k.parent == k.parent.parent.right:
                u = k.parent.parent.left
                if u.color == "RED":
                    u.color = "BLACK"
                    k.parent.color = "BLACK"
                    k.parent.parent.color = "RED"
                    k = k.parent.parent
                else:
                    if k == k.parent.left:
                        k = k.parent
                        self.right_rotate(k)
                    k.parent.color = "BLACK"
                    k.parent.parent.color = "RED"
                    self.left_rotate(k.parent.parent)
            else:
                u = k.parent.parent.right
                if u.color == "RED":
                    u.color = "BLACK"
                    k.parent.color = "BLACK"
                    k.parent.parent.color = "RED"
                    k = k.parent.parent
                else:
                    if k == k.parent.right:
                        k = k.parent
```


G.231.24.0001_Rikabaguswijaya_Prak2_AKA.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect
k = k.parent.parent
else:
if k == k.parent.right:
k = k.parent
self.left_rotate(k)
k.parent.color = "BLACK"
k.parent.parent.color = "RED"
self.right_rotate(k.parent.parent)
self.root.color = "BLACK"
# ===== FUNGSI ANALISIS =====
def get_height(self, node):
if node == self.TNULL:
return 0
return 1 + max(self.get_height(node.left), self.get_height(node.right))
def count_nodes(self, node):
if node == self.TNULL:
return 0
return 1 + self.count_nodes(node.left) + self.count_nodes(node.right)
===== SIMULASI =====
Variables Terminal
Type here to search
21:51
21/04/2026


G.231.24.0001_Rikabaguswijaya_Prak2_AKA.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect
# ===== SIMULASI =====
# WORST CASE (data terurut)
tree1 = RedBlackTree()
for i in range(1, 101):
tree1.insert(i)
h1 = tree1.get_height(tree1.root)
n1 = tree1.count_nodes(tree1.root)
print("WORST CASE")
print("Node:", n1)
print("Height:", h1)
print("log2(n):", math.log2(n1))
print("2log2(n):", 2 * math.log2(n1))
# AVERAGE CASE (data acak)
tree2 = RedBlackTree()
data = list(range(1, 101))
random.shuffle(data)
for x in data:
Variables Terminal
Type here to search
21:51
21/04/2026


G.231.24.0001_Rikabaguswijaya_Prak2_AKA.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Connect
data = list(range(1, 101))
random.shuffle(data)

for x in data:
    tree2.insert(x)

h2 = tree2.get_height(tree2.root)
n2 = tree2.count_nodes(tree2.root)

print("\nAVERAGE CASE")
print("Node:", n2)
print("Height:", h2)
print("log2(n):", math.log2(n2))
WORST CASE
Node: 100
Height: 11
log2(n): 6.643856189774724
2log2(n): 13.287712379549449

AVERAGE CASE
Node: 100
Height: 8
log2(n): 6.643856189774724
Variables
Terminal
Share
Type here to search
21:51
21/04/2026